

Assignment 2:
Software Design Specification
Clothing Store Point-of-Sale System

Group 17:
Andres Ortiz
Max Winchester
Krish Patel

Overview

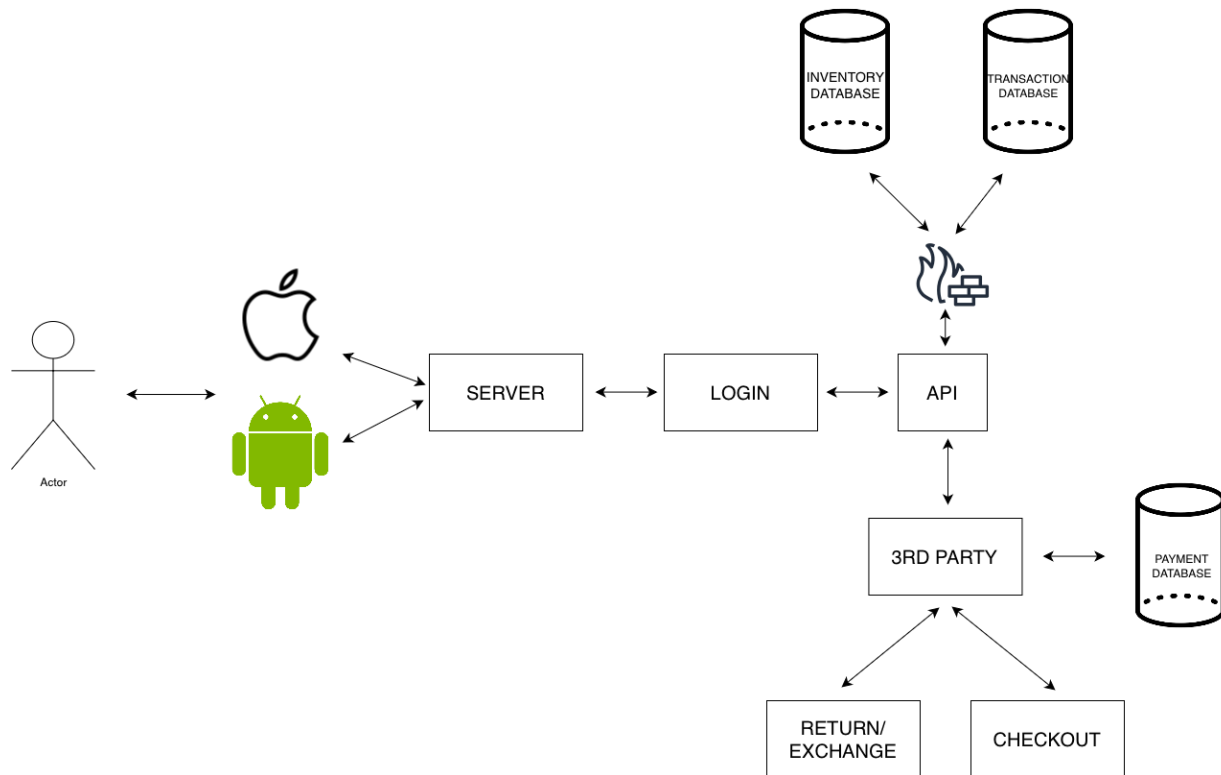
The point-of-sale system is designed not only to assist employees with sales and refunds but also to allow them to manage and update the store's inventory. Employees will have access to the system as well as the inventory database. Administrators will also be able to access a separate, secure transaction database. Both the inventory and the transaction databases will be backed up through a cloud and synchronized with the individual store locations.

The system will utilize external systems to process credit card and debit card transactions. A built-in barcode scanner and sorting system will allow employees to search for items in the database using unique ID codes for each item. Items are also traceable through specific attributes such as color, size, price, etc. The inventory is automatically updated after any transactions.

The system will be supported by both Apple and Android operating systems, and all devices will be connected to the online shared inventory. Through all these components, the overall objective of the system is to make the employees' jobs easier while simultaneously improving efficiency and organization within the stores. This, in turn, will increase sales and also provide maximum customer satisfaction.

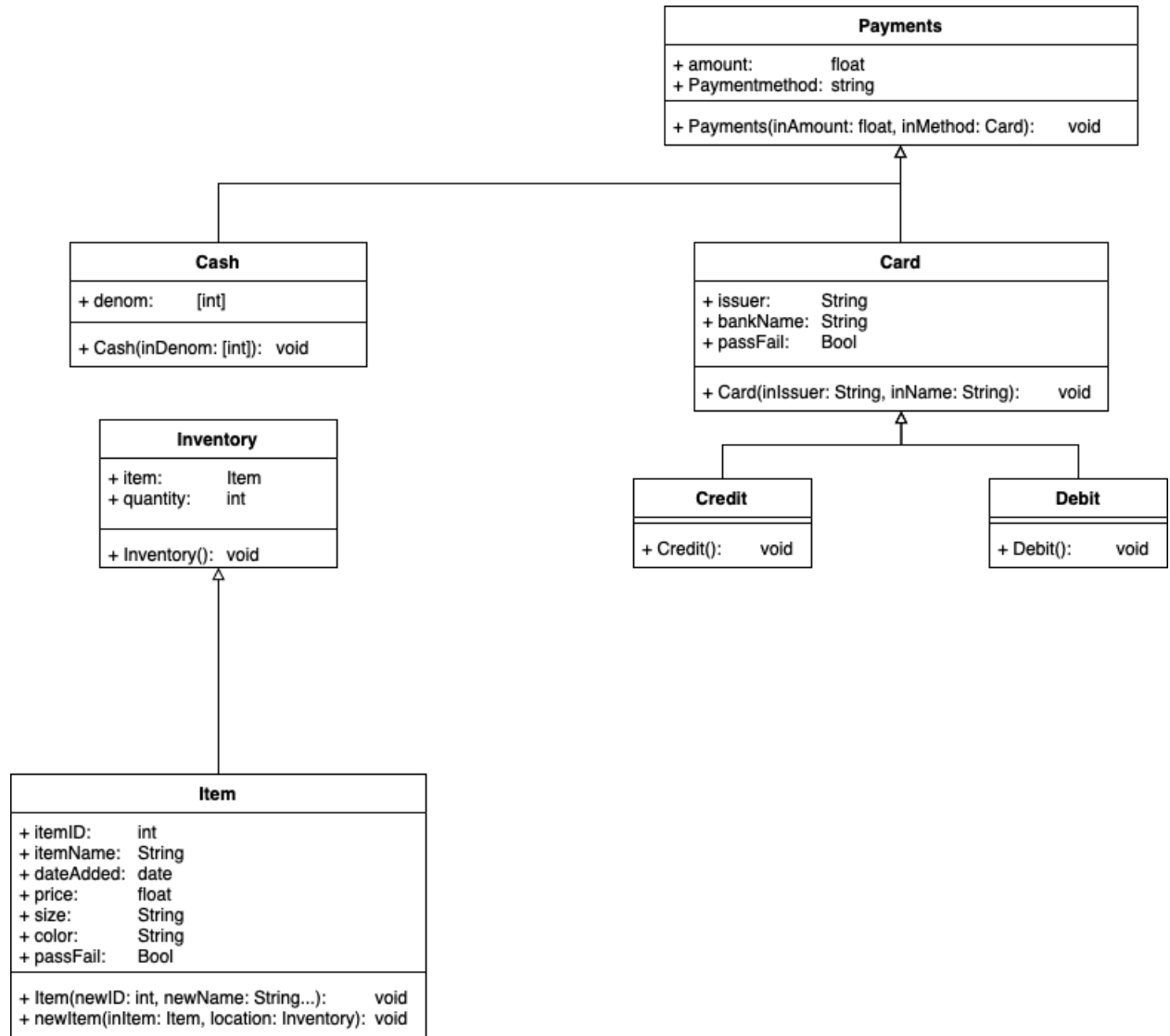
Next, we will review the architecture of all major components, the basic structure of the system, and its primary functions. We will then move to the system's development plan and estimated timeline according to current specifications. Finally, we will discuss the breakdown of work and any updates since the previous submission.

Architectural Diagram



Description: The actor interacts with the server using an Android or Apple device. The server prompts them to log in, which connects them to the API upon a successful login attempt. The API uses a firewall to prevent unauthorized access attempts to the inventory and transaction databases. As for actual transactions, the API connects to a third-party API, which then links to a payment database. It also allows the actor to either do a checkout or a return/exchange.

UML Diagram



Description: The UML has two separate portions. One is for the payments, and the other is for the inventory. The Inventory section is simple; there is an Inventory class with the item and quantity attributes. The item attribute is an 'item' type object, and the quantity is simply an int. The class also has an Inventory() method with a void return type used merely to update the existing inventory. The item object has the following attributes: itemID (int), itemName (String), dateAdded (date), price (float), size (String), color (String), and passFail (Bool). All the other attributes are self-explanatory, but the passFail attribute gets its' value from the methods with a void return type, but they change the value of passFail to true if the function was successful. The methods are Item, which takes in a new ID and Name for the new item, and the newItem method, which takes in the item itself as an object and a location, the Inventory database.

The payment portion has a Payments class with two attributes, amount (float) and paymentMethod (String). It also has a Payments function, which takes in the amount and the Card object. The Card object has three attributes: issuer (String), bankName (String), and passFail (Bool). It also has the Card function, which takes in the issuer and the name with a void return type. The card function then moves to either the Credit or Debit method, which simply completes the transaction, updates the value of passFail, and has a void return type.

Description of Classes

- Payments

- This class contains an amount as a float and a string that determines the payment method. It also has a function with two branches, with separate functions for cash and card.

a. Cash

- This class contains an array of integers that determines individual notes of cash as each element in the array. The class also has the Cash function, which is used to complete the transaction in the system.

b. Card

- This class contains two strings: the bank's and issuer's names. It also has a bool that updates to True if the transaction was successful. Depending on these three arguments, the class uses its Card function, which branches to either the Credit or Debit function.

- Credit

- This function makes the transaction through the credit card company.

- Debit

- This function makes the transaction through the bank connected to the debit card.

- Inventory

- This class contains the quantity as an integer and an item subclass. It also has an inventory function that updates the inventory.

- Item (subclass)

- The Item subclass contains the following attributes:
 - itemID (int)
 - itemName (String)
 - dateAdded (int) → INT instead of date in Diagram?
 - price (float) → FLOAT instead of Currency in Diagram?
 - size (String)
 - color (String)
- It also contains the Item() function.

Description of Attributes

- **Payments:**
 - a. **Amount**
 - This attribute contains a float that determines the amount to be paid by the user, either cash or card.
 - b. **Payment Method**
 - This attribute contains a string containing either “cash” or “card” to tell the system which payment the user will use.
- **Cash:**
 - a. **Denom**
 - This attribute contains an array of integers denominating the amount of currency being received (dimes, nickels, quarters, etc.).
- **Card:**
 - a. **Issuer**
 - This attribute contains a string that holds the card issuer’s information to proceed with a transaction.
 - b. **Bank Name**
 - This attribute contains a string that holds the name of the bank’s card to be used to make the transaction.
 - c. **passFail**
 - This attribute holds a False value until the transaction is made; if the transaction is successful, the value is updated to True, letting the program know that the transaction was completed.
- **Inventory:**
 - a. **Item**
 - This attribute contains information about an item from the inventory that the system can view or modify.
 - b. **Quantity**
 - This attribute contains an int that holds the total amount of inventory from a particular item.

- **Item:**
 - a. **Item ID**
 - This attribute contains an int that holds an item's identification number or SKU, which can be looked up manually or scanned.
 - b. **Item Name**
 - This attribute contains a string with the name of the searched item.
 - c. **Date added**
 - This attribute contains a string that holds the date the item was added or modified to the system.
 - d. **Price**
 - This attribute contains the total cost of an item
 - e. **Size**
 - This attribute contains a string array that holds the available sizes of an item.
 - f. **Color**
 - This attribute contains a string array that holds the available colors of an item.

Description of Operations

Constructors

While our UML diagram omits the getter and setter methods that are required for each class to operate, we have made sure to show constructor methods in each class and subclass. The class uses these constructor methods to initialize attribute variables with input parameters and other pre-set data.

Creating Options

One of the most significant contributors to the list of operations is the client's desire to provide options to their customers. There will be some operations used much more frequently than others, but when dealing with multiple choices of payment method, receipt handling, printing options, and others, all must be kept in mind when in the development stage; an error should not be thrown if no card is provided, for example, if the user has chosen to pay via cash.

Other Operations

Of course, the system will have many more operations than those listed here. Examples include methods for navigating the user interface, handling payment method APIs for digital payments, and controlling and managing the ticketing database, which provides screening information and also prevents shows from being overbooked. These are all critical features of the POS system and must be thoroughly tested to ensure seamless operation and ease of use for the client and their customers.

Delegation of Work

In total, one outline, two diagrams, and three descriptions were needed for this SDS, which were quickly divided into three sets of two tasks each. The tasks were delegated as follows:

Overview	–	Krish
Architectural Diagram	–	Andres
UML Diagram	–	Max
Classes desc.	–	Krish
Attributes desc.	–	Andres
Operations desc.	–	Max
Diagram Descs.	–	Krish

This allowed everyone to have an equal share of the workload.

Assignment 3:

Test Plan

Test Set 1 – Payment

- Unit
 - Testing if card transactions go through correctly.
 - The inputs are the credit card/debit card details, the purchased item, and the price.
 - The function itself makes the transaction through the third-party bank API using the card information.

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int main() {
5
6     float amount = 9.99
7     Card myCard;
8     myCard.passFail = False;
9     myCard.issuer = "VISA"
10    myCard.bankName = "Bank of America"
11
12    Payments(amount, myCard)
13
14    if (myCard.passFail == True){
15        return PASS
16    } else{
17        return FAIL
18    }
19
20 }
21 }
```

- The output is either a pass or fail.
- Functional
 - Testing that the system correctly accepts cash.
 - Various denominations of cash should be input — less than needed, exact change, more than needed — to see whether the program accurately calculates the differences between input and required and will tell the user what else to do.
- System
 - The user scans the items they want to purchase and then chooses the desired payment method (cash or card) they will use. After proceeding to confirm the purchase, a receipt is printed with the total amount due and paid to verify the payment was successful.

Test Set 2 – Inventory

- Unit

- This unit test checks to see if a new item can be successfully added to the inventory. It creates a new Item object, adds the attributes, and then runs the “newItem” function, which takes in the newly created item and a constant location (Inventory).

```
2  #include <stdio.h>
3
4  int main() {
5      Item myItem;
6      myItem.itemID = 123456789;
7      myItem.itemName = "Plain T-Shirt";
8      myItem.dateAdded = "March 14, 2024";
9      myItem.price = 9.99;
10     myItem.size = "Medium";
11     myItem.color = "Black";
12     myItem.passFail = False;
13
14     newItem(myItem, Inventory);
15
16     if (passFail == True){
17         return PASS;
18     } else {
19         return FAIL;
20     }
21
22
23 }
```

- Functional

- Testing to see if newly added items are recognized by the system.
- After a new item is added, the system should then be able to output that item on request now that it has been added to the inventory. If this is not possible, then a check must be made to see whether the issue is with adding the item or instead with the acknowledgment of a change to the inventory list.

- System:

1. **Edit Inventory:** The employee logs in with their credentials and chooses whether they want to edit inventory, add to inventory, or remove from inventory. After the employee selects an option, they make changes, which are applied, and the inventory is updated accordingly.
2. **Purchase/Return:** Employees log in with their credentials, then choose the checkout or return item(s). After the employee selects an option, items are scanned, and the total transaction amount is either charged or returned to the desired payment method (cash or card). Then, a confirmation receipt is printed, and the inventory is updated accordingly.

Updates Since Assignment 2

The changes made to our original SDS are not extensive, as we were pleased with its current state. The main thing we changed was that we added descriptions to the diagrams, as we had not included them before.

The few other changes we have made since the submission for assignment two are mostly grammar and wording improvements, with little change in meaning or content.

Other changes we made were to the UML diagram. We adjusted some of the method and attribute names and types to reflect better the test cases we wanted to create for assignment three: nothing particularly drastic or noticeable, instead just consistency details.